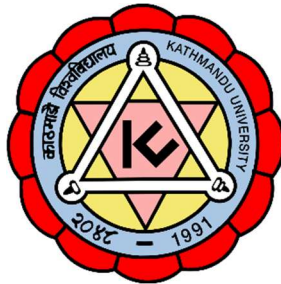


Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



“ImGrep”
A Project Report
Submitted To:
Kathmandu University
Department of Computer Science and Engineering
(In partial fulfillment of the requirements for the Bachelor in Computer Science II/II)
(COMP 207)

Submitted By:
Bijan Bakabal Thapa (7)
Manish Bhattarai (11)
Sadit Rasaili (40)
Sameep Kakshapati Shrestha (20)
Slok Pradhan (38)

Submitted To:
Mr. Suman Shrestha
Department of Computer Science and Engineering
Submission Date: 2025/08/05



Kathmandu University
Department of Computer Science and Engineering
School of Science

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by "Team ImGrep" entitled "ImGrep" in partial fulfillment of the requirements for the Bachelor of Computer Science is recommended for the final evaluation.

SIGNATURE

Mr. Dhiraj Shrestha

Department of Computer Science and Engineering

Supervisor

Acknowledgement

We would like to express our special thanks of gratitude to the Department of Computer Science and Engineering (DOCSE) who allowed us to do this wonderful project "ImGrep", which will help us to familiarize ourselves with the new way of tracking classroom resources, using the web application which helps in the betterment of education and teaching organizations.

Secondly, we would also like to thank our project coordinator Mr. Dhiraj Shrestha for approving our project and for his assistance and guidance. Also, we extend our sincere esteems to all mentors for their timely support soon.

Sincerely,

Team ImGrep

Abstract

In an era marked by the proliferation of digital photography, individuals often accumulate thousands of images across their personal devices without proper labeling or categorization. This unstructured data accumulation creates significant challenges in locating specific images when needed. To address this problem, we developed an intelligent mobile application that enables natural language-based search across a user's photo gallery. The application, built using Flutter for the frontend and Python Flask for the backend, leverages several machine learning and deep learning techniques to analyze and index images semantically. Object detection is handled by ResNet-based models, while Convolutional Neural Networks (CNNs) are employed to perform Optical Character Recognition (OCR) on images, enabling the identification of textual content. Facial recognition further augments image context by detecting and encoding human faces. All visual and textual embeddings are indexed in a FAISS vector store, enabling efficient similarity search, while structured metadata is stored in a PostgreSQL (Neon) database.

This hybrid approach allows the app to process images and associate them with rich semantic vectors and attributes, which are then matched against user queries using natural language processing (NLP) techniques. For instance, if a user searches for “cake with candles,” the application retrieves images where cakes and candles are visually identified or referenced in extracted text. The system provides a responsive and intuitive user interface, allowing users to search their gallery using descriptive phrases, object names, or visible text content. The significance of this project lies in its potential to bridge the gap between human language and raw image data, providing a seamless and intelligent method for personal content management. This solution is especially useful in contexts where manually tagging or categorizing each image is impractical.

Keywords: image retrieval, deep learning, object detection, optical character recognition, facial recognition, ResNet, CNN, FAISS, image embeddings, Flutter, natural language search, PostgreSQL, Flask backend, unstructured data management

Table of Contents

Acknowledgement	iii
Abstract	iv
Table of Contents	vi
List of Figures	viii
Acronyms/ Abbreviations	ix
Chapter 1: Introduction	10
1.1. Background:	10
1.2. Objectives	11
1.3. Motivation and Significance	11
1.4. Expected Outcomes	12
Chapter 2: Related Works	14
2.1. Google Photos	14
2.2. Samsung Gallery	15
Chapter 3: Methodology	16
3.1. Development Phases	16
3.2. System Requirements	21
3.2.1. Software Specifications	21
3.2.2. Hardware Specifications	25
Chapter 4: Accomplishments	26
Chapter 5: System Walk-through	31
Chapter 6: Conclusion and Recommendations	35
7.1. Limitation	35

7.2. Future Enhancements.....	36
REFERENCES	38
APPENDIX A: GANTT CHART.....	39
APPENDIX B: MACHINE LEARNING.....	40
APPENDIX C: SNAPSHOTS	41

List of Figures

Figure 2.1. Google Photos Search Page UI	14
Figure 2.2. Samsung Gallery Search Page UI.....	15
Figure 3.1. Schematic Diagram of Database	22
Figure 3.2. Working of CNN	23
Figure 3.3. Skip Connection in Resnet	24
Figure 4.1. Use Case Diagram	30
Figure 5.1. User Interaction Flow	31
Figure 5.2. Home Page Flow	32
Figure 5.3. Search Page Flow	33
Figure 5.4. Library Page Flow	34
Figure A. Gantt Chart	39
Figure B.1. Error on 20 vs 56 layer CNN model	40
Figure B.2. Plain Neural Network vs Residual Network	40
Figure C.1. Get Started Page.....	41
Figure C.2. Library Page.....	41
Figure C.3. Meta Data and Caption View Page.....	42
Figure C.4. Image Viewer Page.....	42
Figure C.5. Search Page.....	43

Acronyms/ Abbreviations

i.	AI	Artificial Intelligence
ii.	API	Application Programming Interface
iii.	CNN	Convolutional Neural Network
iv.	CLIP	Contrastive Language–Image Pretraining
v.	FAISS	Facebook AI Similarity Search
vi.	GPU	Graphics Processing Unit
vii.	NLP	Natural Language Processing
viii.	OCR	Optical Character Recognition
ix.	PyTorch	Python Torch
x.	ReLU	Rectified Linear Unit
xi.	REST	Representational State Transfer
xii.	SQL	Structured Query Language
xiii.	UI	User Interface

Chapter 1: Introduction

1.1. Background:

In today's digital world, individuals capture and store thousands of images on their personal devices, leading to an ever-growing volume of unstructured visual data. While smartphones and cloud services make it easy to preserve these memories, retrieving a specific image from such large galleries often becomes tedious—especially when users are unsure of the date, location, or filename associated with the image. This challenge highlights the growing need for intelligent image search and organization solutions that go beyond traditional folder or date-based systems.

Recent breakthroughs in artificial intelligence, particularly in computer vision and deep learning, have enabled automated understanding of image content. Convolutional Neural Networks (CNNs), such as the ResNet architecture, have proven highly effective in object recognition tasks by learning to identify complex visual patterns and features. These models allow systems to detect and classify objects within an image without human intervention, offering a powerful foundation for content-aware image retrieval.

Another key advancement is Optical Character Recognition (OCR), which enables the extraction of textual information from images. With OCR, systems can recognize and process printed or handwritten text in images of documents, signs, and other sources, making it possible to search images using textual queries. This technique is essential in applications where text within the image holds contextual or semantic importance.

Beyond object and text recognition, additional techniques such as **facial recognition** allow the system to detect and group images of people, while **metadata extraction** (e.g., timestamps, geolocation, device data) adds further context to each image. These combined capabilities transform a simple image gallery into a searchable, intelligent system where users can enter natural language queries and receive relevant results, even when the images themselves have no prior labels.

The goal of this project is to design and implement an application that leverages these modern AI technologies to automatically sync images from the user's gallery and enable efficient search using visual and textual content. By integrating object recognition, OCR, facial recognition, and metadata extraction, the application aims to offer a seamless and intuitive user experience that significantly improves image organization and retrieval.

1.2. Objectives

The primary objectives of the ImGrep project are:

- i. To perform object recognition on the images using a deep learning model based on ResNet, enabling detection and tagging of visual content.
- ii. To enable query-based image retrieval by matching user-entered search terms against both recognized objects and extracted text.
- iii. To implement Optical Character Recognition (OCR) for detecting and extracting textual content from images to support search functionality.
- iv. To extract metadata of images to optimize the searching feature in the app.
- v. To synchronize images from the device's gallery into the application for analysis and indexing.

1.3. Motivation and Significance

With the growing volume of images stored on personal devices, the need for intelligent and efficient methods of organizing and retrieving visual information has become increasingly important. Users often find it difficult to locate specific images based solely on memory or manual browsing, especially when the gallery contains hundreds or thousands of photos. Traditional gallery applications offer limited support for content-based search, relying mostly on metadata such as date, location, or user-created albums. This limitation motivates the development of an application that can provide deeper, semantic-level image understanding and search.

The core motivation behind this project is to bridge that gap by enabling users to perform query-based searches across their gallery using our specialized hybridized model. By recognizing the content of images, whether it is an object like a “laptop” or text such as a “receipt” the application enhances usability and accessibility. This is particularly useful in educational, organizational, and archival contexts, where users frequently need to retrieve images based on specific visual elements or embedded text.

The significance of this work lies in its integration of deep learning technologies, such as CNNs and ResNet, into a user-friendly mobile application. The system’s ability to recognize and index underlying image features empowers users with a more intelligent image retrieval system compared to conventional tools. Moreover, by using technologies like Flutter for cross-platform development and Faiss for fast vector search, the application remains efficient, scalable, and suitable for real-world deployment.

This project demonstrates the practical application of machine learning and mobile development in solving everyday user problems. It also provides a foundation for future extensions, such as emotion tagging, or integration with cloud-based workflows, highlighting its potential academic and commercial impact.

1.4. Expected Outcomes

The development and deployment of the proposed image recognition and search application are expected to yield several key outcomes:

- i. **Efficient Image Synchronization:** The application will successfully retrieve and store images from the device’s gallery, maintaining local or cloud-based access for further processing.

- ii. **Accurate Object Recognition:** Through the use of deep learning models such as ResNet, the system is expected to accurately identify and classify common objects present in user images, enabling content-aware search functionality.
- iii. **Effective Text Detection via OCR:** The Optical Character Recognition (OCR) module will extract readable text from images with a high degree of accuracy, supporting text-based queries for document scanning, signage recognition, or general image-text retrieval.
- iv. **Responsive Query-Based Search:** Users will be able to input keywords or phrases, and the system will return relevant images by matching the query against recognized objects and extracted text, significantly reducing the time and effort needed to locate specific images.
- v. **User-Friendly Mobile Interface:** The Flutter-based interface is expected to provide a smooth, intuitive user experience with cross-platform compatibility, making the application accessible on both Android and iOS devices.
- vi. **Scalable and Efficient Backend Processing:** The Python (Flask) backend, integrated with Faiss and PostgreSQL (Neon), will enable fast, scalable, and reliable image indexing and retrieval, supporting real-time interactions without compromising performance.

Overall, the application aims to deliver a practical, intelligent solution for managing and searching personal image collections, demonstrating the effectiveness of combining machine learning with mobile development for real-world tasks.

Chapter 2: Related Works

2.1. Google Photos

Google Photos is a cloud-based photo management and storage application developed by Google. It leverages powerful machine learning and computer vision techniques to automatically analyze and categorize images. The app can recognize faces, objects, scenes, landmarks, and even text using OCR, allowing users to search their image library using natural language queries. For instance, entering terms like “dog,” “birthday,” or “Eiffel Tower” returns relevant photos without manual tagging. Google Photos also supports automatic album creation, facial clustering, timeline-based browsing, and intelligent suggestions. Its cloud integration allows users to back up and sync photos across devices seamlessly. The combination of storage, organization, and intelligent search makes it one of the most comprehensive photo management tools currently available.

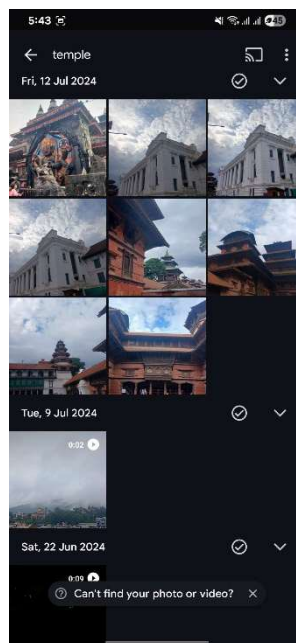


Fig 2.1. Google Photos Search Page UI

2.2. Samsung Gallery

Samsung Gallery is the default photo viewing and management application pre-installed on Samsung devices. While it focuses primarily on local photo organization, recent versions of the app incorporate AI-based features for automatic content recognition and categorization. Samsung Gallery can identify various elements in images, such as food, landscapes, people, pets, and documents, and automatically group them into categories for easier browsing. It offers features such as face recognition, timeline view, tagging, and integration with Samsung Cloud and Microsoft OneDrive for cloud backup. The app emphasizes privacy and on-device processing, allowing image recognition features to work without needing an internet connection. Its seamless integration with the Samsung ecosystem makes it a popular choice among Galaxy users.

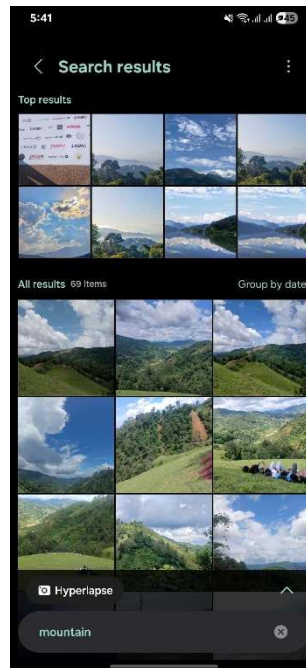


Fig 2.2. Samsung Gallery Search Page UI

Chapter 3: Methodology

The development of the Lobic project adhered to a structured and methodical approach, consisting of well-defined phases that ensured the creation of a robust and user-friendly music streaming platform. Modern development methodologies, such as Agile practices, were employed to foster flexibility, adaptability, and continuous iterative improvement throughout the development lifecycle. This approach enabled the team to effectively respond to user feedback and evolving requirements, ensuring the delivery of a high-quality application.

3.1. Development Phases

i. Planning and analysis

During the planning and analysis phase, the focus was on exploring and understanding the foundational concepts and technologies necessary to build an intelligent image search application. At this stage, the exact tools and models to be used had not yet been finalized. Instead, the team concentrated on studying how object recognition systems function, particularly how convolutional neural networks (CNNs) are able to detect and classify visual features in images. This included learning about architectures such as ResNet, which introduced the concept of residual learning to enable the training of deep models without performance degradation.

The team also examined the role of **embeddings** in machine learning, specifically how high-dimensional vector representations can be used to capture semantic similarities between images and text. This led to discussions on how such embeddings could support similarity-based image retrieval, forming the basis for implementing a searchable index. In addition to visual recognition, the application's ability to handle text-based queries required a basic understanding of **natural language processing (NLP)** techniques. This included how text input

from users could be interpreted and matched against object labels or extracted text from images.

A significant part of the analysis involved assessing the **feasibility** of implementing these components in a mobile environment. Questions were raised about whether tasks like object detection and optical character recognition (OCR) could be handled on-device or would require a backend server. Considerations included performance, latency, model size, and user experience. The planning phase was thus marked by active research, experimentation, and technical evaluation, laying the groundwork for the later design and implementation phases of the project.

ii. **Research and Learning**

Following the planning and analysis phase, the next stage of the project involved extensive research and learning to identify suitable technologies and deepen our understanding of the key components required to implement the system. This phase focused on evaluating existing tools and machine learning models, comparing their capabilities, and determining how they could be integrated into the application to achieve the desired functionality.

One major area of exploration was the use of **CLIP (Contrastive Language–Image Pretraining)**, a model developed by OpenAI that connects images and text in a shared embedding space. CLIP was particularly relevant to our goal of enabling query-based image search, as it allows images and natural language descriptions to be compared directly using vector similarity. The team studied how CLIP could be used to generate embeddings for both visual and textual data, making it a potential backbone for the search functionality.

We also investigated **image captioning** and various **datasets**, such as MS COCO and ImageNet, to understand how labeled data supports model training and evaluation. This helped us evaluate whether we should use **pretrained models** or

build our own **custom models** for object detection and classification. At this stage, the trade-offs between using off-the-shelf models and training domain-specific ones became clearer.

For text recognition, we explored multiple OCR libraries. **Tesseract OCR**, an open-source engine originally developed by HP and now maintained by Google, was tested for its accuracy, speed, and multilingual support. In parallel, we experimented with **EasyOCR**, a deep learning-based library that provides better recognition in more complex or non-standard visual scenarios. These tools were evaluated to determine which would best meet the requirements for recognizing text embedded in user images.

Additionally, we studied **Faiss (Facebook AI Similarity Search)** for implementing a fast and scalable similarity search engine based on image and text embeddings. Faiss supports high-dimensional vector indexing and was identified as an effective solution for retrieving semantically similar images based on user queries. Finally, the team explored **Duckling**, a natural language processing library developed by Facebook for parsing and understanding structured entities like dates, times, and quantities. Although not central to the initial version of the app, Duckling was considered for future features that might involve intelligent query interpretation.

Overall, this research and learning phase was critical in informing our technology choices and building the foundational knowledge necessary to proceed with the implementation. By exploring both classical and state-of-the-art tools across computer vision, OCR, embedding search, and NLP, we were able to make well-informed decisions about the architecture and components of the final system.

iii. Development

The development phase marked the transition from research and planning to the practical implementation of the system. Drawing upon insights gained during earlier stages, the team adopted a technology stack combining mobile development frameworks, backend services, and machine learning libraries to build a fully functional application capable of synchronizing images, performing recognition tasks, and supporting efficient search.

The **frontend** of the application was developed using **Flutter**, enabling cross-platform compatibility on Android and iOS devices from a single codebase. Flutter provided a responsive and modern user interface, including features for browsing synced images, entering search queries, and viewing search results. To support offline functionality and local caching of image metadata, **SQFlite** was integrated as the embedded database layer on the device.

On the **backend**, a REST API was developed with **Python (Flask)** to handle operations such as image processing, embedding generation, and search queries. The backend received images and metadata from the mobile app, processed them using deep learning models, and returned relevant results. This server-based approach allowed more computationally intensive tasks, such as running large neural network models, to be performed without impacting the responsiveness of the mobile interface.

Object recognition was implemented using **ResNet** models trained in **PyTorch**. The ResNet architecture enabled accurate detection and classification of objects within images by leveraging pretrained weights fine-tuned on relevant datasets. When an image was processed, the model generated predictions corresponding to recognized objects, which were stored as labels and embeddings. To support text-based search, **Optical Character Recognition (OCR)** capabilities were added using both **Tesseract OCR** and **EasyOCR**. This allowed the system to detect and

extract text content embedded in images, such as signs, receipts, or printed documents.

All recognized objects and text were transformed into **vector embeddings** to enable similarity search. The team integrated **Faiss (Facebook AI Similarity Search)** to index these high-dimensional vectors efficiently. When a user submitted a query, the backend generated an embedding representation of the text and used Faiss to retrieve the nearest matching images based on vector distance. This approach provided fast and accurate search performance even across large collections of images.

In parallel, **PostgreSQL (Neon)** was used to store structured information about images, users, and query logs. The combination of a relational database for metadata and Faiss for vector similarity enabled both precise and semantic search capabilities. During development, care was taken to design secure API endpoints, manage authentication, and ensure that user data remained private and protected.

Throughout the development process, iterative testing was performed to evaluate recognition accuracy, OCR performance, and search relevance. Usability testing on real devices informed refinements to the interface and interaction flow. The integration of all components, image synchronization, object detection, OCR, embeddings, similarity search, and user interface, resulted in a cohesive system that met the objectives defined in the earlier phases of the project.

iv. Documentation

The documentation phase focused on recording every major aspect of the application's development, structure, and functionality to ensure clarity, maintainability, and reproducibility. Proper documentation is essential not only for future development and scaling but also for enabling other developers, collaborators, or users to understand the system's design and usage.

During this phase, detailed explanations of the **project architecture**, **technology stack**, and **data flow** were compiled. Each module—such as the image synchronization component, object recognition system, OCR engine, embedding generation process, and query handling logic—was documented with clear descriptions of its purpose, inputs, outputs, and dependencies. This allowed the overall system to be understood in a modular and systematic way.

Altogether, this phase ensured that the work done throughout the project was clearly recorded, traceable, and easy to continue or build upon, fulfilling one of the most important long-term aspects of software development: maintainability.

3.2. System Requirements

3.2.1. Software Specifications

The application is built using a combination of mobile and backend technologies to support image synchronization, processing, and intelligent search. The major components of the tech stack are as follows:

3.2.1.1. Front-End Tools

Frontend tools that were used to build this application are:

- i. **Flutter:** Flutter is an open-source UI toolkit developed by Google for building natively compiled applications from a single codebase. It was used to develop the mobile interface of the application, offering cross-platform support, responsive design, and smooth user experience.
- ii. **SQFlite:** SQFlite is a Flutter plugin that provides an interface for using SQLite, a lightweight, embedded SQL database. It is used in the application for local storage, caching of images, metadata, and quick retrieval of user-specific data offline.

3.2.1.2. Back-End Tools

Backend tools that were used to build this application are:

- i. **Python (Flask):** The backend is developed using Flask, a micro web framework in Python. Flask handles API requests from the mobile app, processes image data, and communicates with the database. It provides a lightweight and flexible environment for interacting with machine learning models and serving responses efficiently.

3.2.1.3. Database

Database system that were used to build this application are:

- i. **Faiss:** Faiss (Facebook AI Similarity Search) is a library developed by Facebook AI Research for efficient similarity search and clustering of dense vectors. In the application, Faiss is used to index image and text embeddings, enabling fast and accurate retrieval of images based on content similarity.
- ii. **PostgreSQL (Neon):** PostgreSQL is a powerful, open-source relational database system. Neon is a serverless and cloud-native implementation of PostgreSQL that allows scalable and fast storage without managing infrastructure. It is used to store structured data such as user information, image metadata, and search logs.

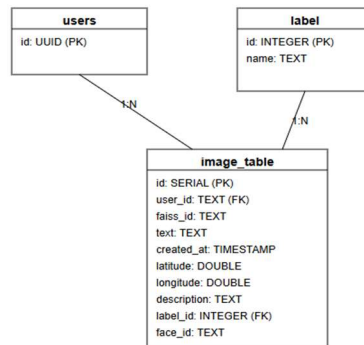


Figure 3.1. Schematic Diagram of Database

3.2.1.4. Machine Learning

Library and models used to build this application are:

- i. **PyTorch:** PyTorch is an open-source deep learning framework widely used for research and production. It is used in the backend for model development and inference tasks related to image features extraction.
- ii. **Convolutional Neural Networks (CNNs):** A standard CNN architecture includes convolutional layers, activation functions like ReLU to introduce non-linearity, pooling layers for spatial downsampling, and fully connected layers for output classification. This combination allows CNNs to generalize well across a wide variety of image recognition tasks, even when inputs vary in lighting, orientation, or background.

CNNs have proven highly effective in tasks like image classification, object detection, facial recognition, and particularly in optical character recognition (OCR). OCR involves detecting and decoding characters from images, often in complex real-world conditions such as handwriting, low lighting, or cluttered backgrounds. CNNs are capable of extracting relevant features from these challenging inputs, making them well-suited for OCR systems. In our application, CNNs are used as part of the pipeline to recognize and extract textual information from synced images. By learning spatial features associated with letters and digits, the CNN enables accurate identification of characters even in noisy or varied visual contexts.

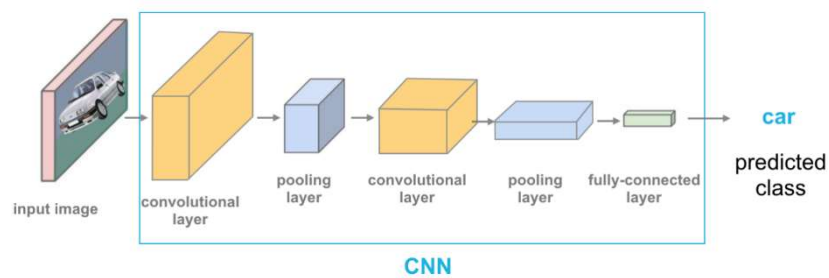


Figure 3.2 Working of CNN

- iii. **ResNet:** ResNet (Residual Neural Network) is a deep learning architecture introduced in 2015, which won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) that year. It marked a major breakthrough in training very deep convolutional neural networks (CNNs).

Traditional CNNs suffer from a problem called the vanishing gradient, where gradients become too small to effectively update weights in early layers as networks get deeper. This leads to degradation of performance, even when more layers are added.

To overcome this, ResNet introduced the concept of residual learning. Instead of each layer learning a direct mapping, ResNet allows layers to learn the residual (or difference) between the input and the output. This is done using skip connections, which bypass one or more layers and directly add the input of a block to its output. Mathematically, if the input is x and the block learns a function $F(x)$, then the output becomes $F(x) + x$.

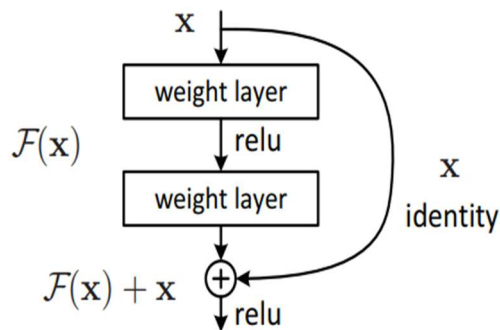


Figure 3.3 Skip Connection in Resnet

3.2.2. Hardware Specifications

This application was developed and optimized to run on the following minimum hardware setup:

- i. Server requirements:
 - Processor: Ryzen 7 (8000 series) / Intel i7 12th Gen
 - Ram: 16 GB
 - Disk: 500 SSD
 - OS: Windows 11 / Any Linux Distribution
 - GPU: Nvidia RTX 4050
- ii. Application requirements:
 - OS: Android 13+
 - Processor: Snapdragon 685
 - RAM: 4 GB
 - Disk: 128 GB

Chapter 4: Accomplishments

A major accomplishment of the ImGrep project was the successful integration of advanced AI models and a unified search pipeline into a fully functional Android application. ImGrep was developed exclusively for Android, which required not only working with Flutter for the UI but also extending functionality through native Kotlin code. Features such as real-time gallery listening and image hooking demanded low-level access to the device's file system and media store, which could not be achieved through Flutter alone. Learning and integrating Kotlin to handle native Android services and linking it back to Flutter through platform channels posed both technical and architectural challenges that we overcame through extensive research and iterative testing.

Another significant achievement was building and training custom models for OCR, object detection, and image captioning rather than relying on third-party APIs. This decision allowed us to maintain full control over model accuracy, performance, and data privacy. Training these models required deep understanding of PyTorch, GPU training, and data annotation. We also implemented face recognition using FaceNet-PyTorch, enabling robust clustering and retrieval of images by individual faces. On the natural language processing side, we combined regex-based query parsing with SpaCy's entity extraction to transform natural user queries into structured filters and embeddings. This allowed us to accept human friendly inputs like "photos with text and two people from last December" and break them down into actionable search criteria.

Developing ImGrep also pushed us to learn and use a broad range of tools and techniques, many of which were new at the start of the project. From building a vector index with Faiss for fast similarity search to managing local caching with Sqflite and creating interactive highlight stories through custom Flutter UI components, the project required us to quickly master new frameworks while balancing performance and usability. One of the standout accomplishments on the frontend was the implementation

of a highlight generation system that dynamically calculates monthly and yearly summaries of the user's photo activity all computed locally on the device. Together, these accomplishments highlight the depth of technical effort, problem-solving, and interdisciplinary learning that went into ImGrep.

Another key accomplishment is the seamless coordination of AI-powered search modalities visual tags, face embeddings, image text (OCR), metadata, and generated captions into a single query pipeline. This allowed users to search their photo collections using natural language or combinations of criteria (e.g., "pictures with two people near the beach in 2023"). The frontend also included a gallery listener that continuously monitored device storage for changes, enabling automatic indexing of new or deleted images without manual refresh. Additionally, a highlight engine was implemented entirely on the client side to compute and visualize monthly and yearly summaries of user photos. These highlights were generated using image timestamps to provide meaningful recaps of each month or year's activity, enriching the user experience and enabling contextual browsing. The following challenges were faced during the making of this project:

i. Real-Time Updates

To ensure immediate indexing of new photos, we integrated a native gallery listener in Kotlin that detected changes in the Android media store. Bridging these updates to Flutter without performance lags was a key technical hurdle.

ii. Database Efficiency

Maintaining sync between local Sqflite and remote PostgreSQL while handling frequent indexing and queries demanded optimized schema design and query execution to ensure responsive performance.

iii. **Adopting New Technologies**

We had to quickly learn and apply unfamiliar tools like Faiss, FaceNet-PyTorch, Flutter, and Kotlin. The steep learning curve added to the development complexity.

iv. **Custom AI Model Training**

Instead of relying on APIs, we trained our own OCR, object detection, and captioning models. This required dataset preparation, model tuning, GPU-based training, and mobile-optimized deployment.

v. **Natural Language Query Parsing**

Interpreting queries like “pictures from last winter with text and two people” required building a custom parser using regex-based logic to extract temporal, visual, and contextual elements.

4.1. Features:

i. **Unified Multimodal Search Engine:**

ImGrep delivers a wide range of features that transform ordinary image galleries into smart, searchable collections powered by custom image processing models and natural language understanding. Central to the application is its unified search engine, which processes a single user query and matches it against multiple layers of information extracted from images: facial features, object tags, OCR text, auto-generated captions, and structured metadata. This multimodal pipeline enables users to perform intuitive searches like “photos with laptops and documents in January” and receive highly relevant results. Query interpretation is powered by a hybrid query parsing system combining regex rules and SpaCy-based entity extraction, enabling flexible and human-friendly sentence input.

ii. **Face Detection and Clustering**

The app includes face detection and recognition using FaceNet embeddings, allowing users to group, filter, and search their gallery by individuals—even without manual tagging. This feature makes it easy to locate images featuring specific people and is especially useful for organizing large, untagged photo collections.

iii. **Object Detection and Visual Tagging**

Powered by a custom-trained object detection model, ImGrep automatically identifies and labels visual elements in each image, such as animals, vehicles, electronics, and everyday items. Users can then search for images based on these detected objects, enabling queries like “images with dogs and backpacks” to return precise results based on actual visual content.

iv. **OCR (Optical Character Recognition)**

ImGrep integrates an in-house trained OCR engine to detect and extract visible text from images, including signs, screenshots, or documents. This allows users to perform text-based searches across their photo collection, retrieving images based on keywords, names, phrases, or codes embedded within the visual content.

v. **Image Captioning**

The app uses a custom image caption generation model to add descriptive context to photos. These auto-generated captions provide semantic insight into images that lack metadata or recognizable text, further enhancing the accuracy and depth of the search engine.

vi. **Metadata-Based Filtering**

Users can refine their searches using image metadata, such as capture time and date ranges. This feature enables highly specific queries, for example: “photos taken of Pixel art in December 2023.”

vii. Real-Time Gallery Listener

A native Kotlin gallery listener continuously monitors the Android device's media storage to detect when new images are added, modified, or deleted. This ensures that the app remains fully synchronized with the user's gallery without requiring manual refresh, providing a seamless and responsive user experience.

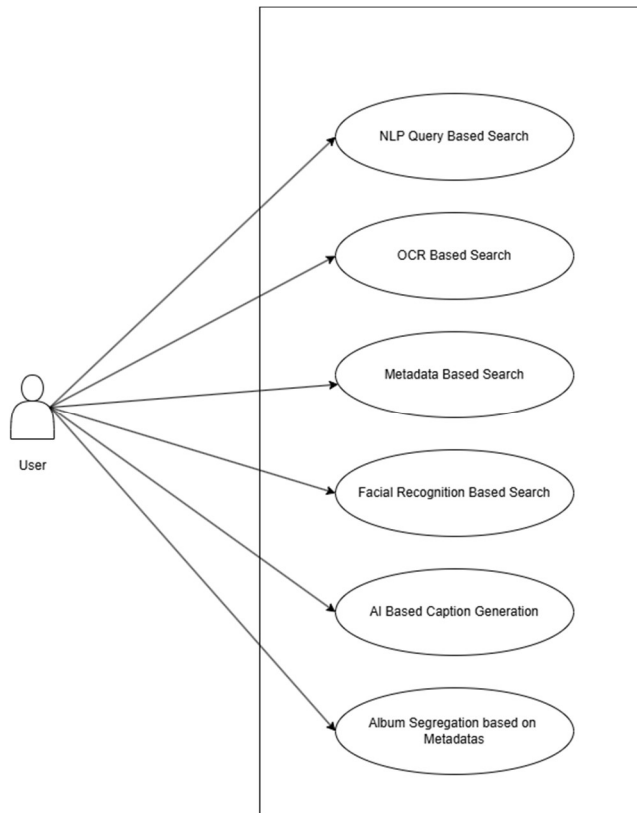


Figure 4.1. Use Case Diagram

Chapter 5: System Walk-through

The detailed walk-through of the developed platform's key features and user interaction is explained below:

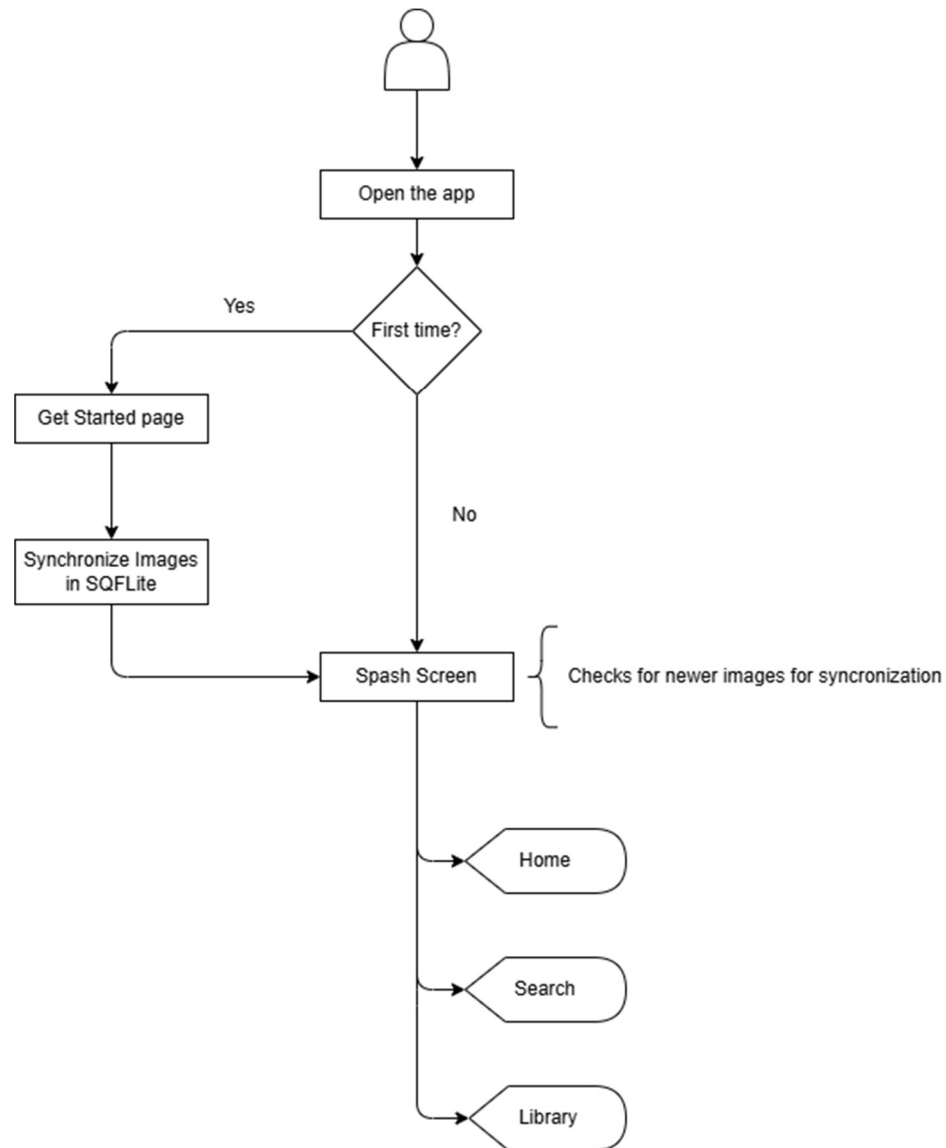


Figure 5.1. User Interaction Flow

1. **Splash Screen:** The splash screen is the first screen that the user arrive as they open the app, the splash screen wraps the loading of the images inside the

database and sync the application with the most recent photos from the local storage.

2. **Get Started Page:** Get started page is the landing page of the application where the users are greeted for the first time and are offered the general overview of the application and its core features.
3. **Home Page:** Home page is the page that has collection of all the images that are present in the device storage or the selected images that the user has allowed to view.

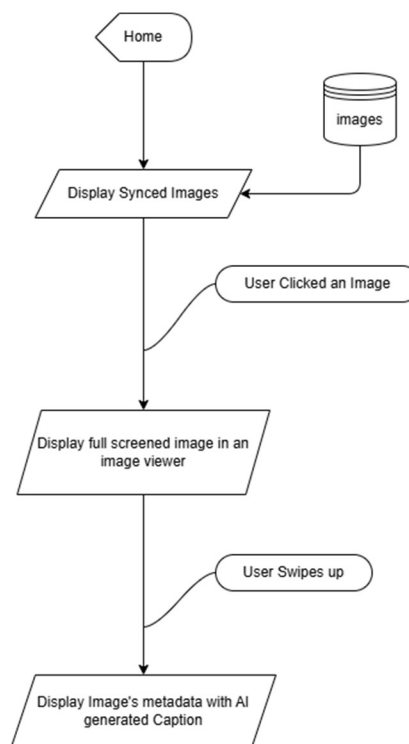


Figure 5.2. Home Page Flow

4. **Image View Page:** Image view page is the page with the one of the feature that consists of view the image and scrolling over the images in the devices, which also allows the user to get information about the meta data of the image and the caption generated from the image

5. **Search Page:** Search page is the page that wraps the core feature of the app that is searching for the image based on the user query, it wraps the aspect of prompt searching including image recognition, metadata, OCR and facial recognition.

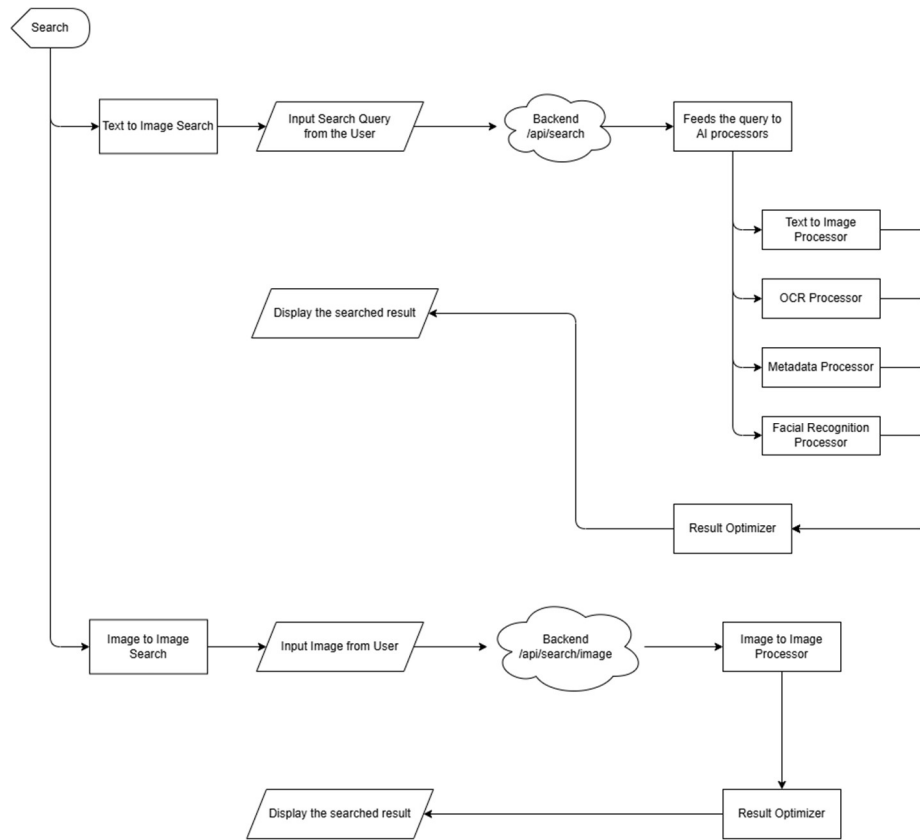


Figure 5.3. Search Page Flow

6. **Library Page:** Library page wraps the feature that creates various albums that are divided into monthly highlights section, yearly highlights section and the facial recognition section that give user the optimal navigation experience.

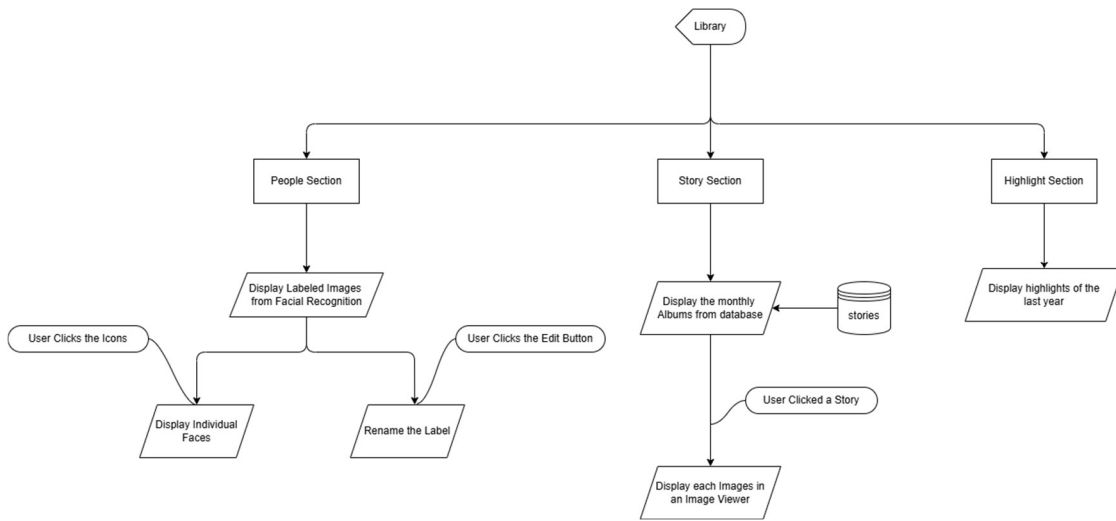


Figure 5.4. Library Page Flow

7. **Sync Page:** The sync page is the image that links the application to the backend which allows the user to upload image to the backend server and initiate the searching of images.

Chapter 6: Conclusion and Recommendations

The ImGrep project redefines traditional photo management and image search by transforming static galleries into intelligent, searchable visual archives. By leveraging custom-trained models for face detection, OCR, object recognition, and captioning all unified under a single query system ImGrep introduces a new paradigm in image retrieval. Unlike conventional gallery apps that depend on filename or timestamp searches, ImGrep enables users to perform natural, sentence-level queries and retrieve images based on people, objects, embedded text, or contextual meaning.

This hybrid search model combines facial recognition, object detection, OCR, and metadata parsing into one cohesive experience, dramatically improving search relevance and flexibility. Through its integration with native Android components (e.g., a Kotlin-based gallery listener), the app ensures real-time synchronization with the device's media store, enhancing usability and responsiveness. ImGrep stands out by offering intelligent automation, rich semantic filtering, and contextual.

7.1. Limitation

While ImGrep introduces powerful features, several limitations exist that offer opportunities for further improvement:

- i. **Local Model Execution:** All AI models including face recognition, OCR, and object detection currently run on a connected local server. This imposes hardware requirements on the user and limits performance scalability. A cloud-backed or hybrid inference approach could help offload compute-heavy tasks and improve cross-device usability.
- ii. **Android-Only Support:** ImGrep is currently supported only on Android. This limits its user base and makes it inaccessible to iOS or desktop users. Extending support to other platforms could broaden adoption and provide a more seamless multi-device experience.

- iii. **No Cross-Device Sync:** The app processes and stores data locally, meaning there's no built-in sync across devices or backup to the cloud. Users may lose data if the app is uninstalled or the device is lost. Cloud integration or encrypted sync services could address this issue.
- iv. **No In-App Tagging or Manual Corrections:** While the app automatically detects faces, objects, and text, it does not yet allow manual correction or user-added tags. Enabling interactive feedback or editing could improve the accuracy of search results and provide a more personalized photo organization system.

7.2. Future Enhancements

To maximize ImGrep potential and enhance the user experience, the following improvements are proposed:

- i. **Cloud Integration and Hybrid Inference:** Introducing cloud-based model inference would reduce hardware dependency and improve performance on lower-end devices. A hybrid approach, with local processing for privacy-sensitive tasks and cloud support for heavier models, could significantly improve scalability and user experience.
- ii. **Multi-Platform Support:** Expanding support to iOS and desktop platforms would allow users to access ImGrep across devices. A web-based dashboard or desktop client could also enable more advanced features such as bulk tagging, detailed analytics, or manual query refinement.
- iii. **Cloud Backup and Sync:** Adding cloud sync functionality will allow users to back up their data, synchronize highlights, and access their photo search history across devices. This is essential for long-term usage, device migration, or collaborative access.
- iv. **User Feedback and Manual Labeling:** Future updates could introduce interactive tagging, allowing users to name people, correct OCR results, or

validate object detections. This would make the search engine smarter over time and allow for the creation of custom, user-defined metadata layers.

- v. **Offline Summaries and Auto-Highlights:** Further improvements to the highlight system such as location-based clustering, trip summaries, or automated event detection could enrich how users revisit their memories. These features could remain fully offline to preserve privacy while enhancing usability.

REFERENCES

- Facebook AI Research. (2017). *FAISS: A library for efficient similarity search and clustering of dense vectors*. GitHub. <https://github.com/facebookresearch/faiss>
- Flutter. (n.d.). *Flutter – Build apps for any screen*. Google. <https://flutter.dev>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- Neon. (n.d.). *Neon serverless Postgres: Scalable, fast, and easy*. <https://neon.tech>
- PostgreSQL Global Development Group. (n.d.). *PostgreSQL: The world's most advanced open source relational database*. <https://www.postgresql.org>
- SQLite. (n.d.). *SQLite: Embedded database engine*. <https://www.sqlite.org>
- Tekartik. (n.d.). *SQFlite – Flutter plugin for SQLite*. pub.dev. <https://pub.dev/packages/sqflite>
- The Pallets Projects. (n.d.). *Flask – Python web framework*. <https://flask.palletsprojects.com>
- GeeksforGeeks. (n.d.). *Introduction to Convolutional Neural Network*. <https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/>
- GeeksforGeeks. (n.d.). *Residual Networks (ResNet) in deep learning*. <https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/>
- Gwardys, G. [Gwardys]. (2020, July 5). *Convolutional Neural Network from scratch (with step-by-step math) [Video]*. YouTube. <https://www.youtube.com/watch?v=Lakz2MoHy6o>
- PyTorch. (n.d.). PyTorch documentation. <https://docs.pytorch.org/docs/stable/index.html>

APPENDIX A: GANTT CHART

Imgrep Application Development Gantt Chart

PROCESS	1	2	3	4	5	6	7	8	9	10
Planning										
Wireframing										
Design Process										
Front-end development										
Back-end development										
Deployment And Documentation										

Figure A: Gantt Chart

APPENDIX B: MACHINE LEARNING

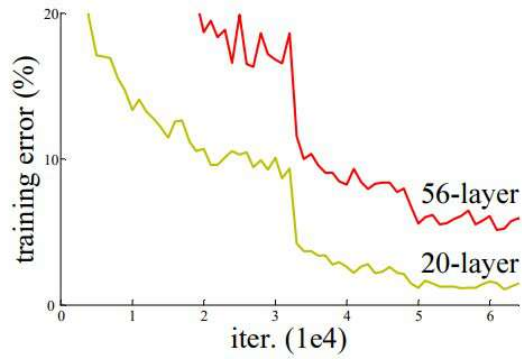


Figure B.1. Error on 20 vs 56 layer CNN model

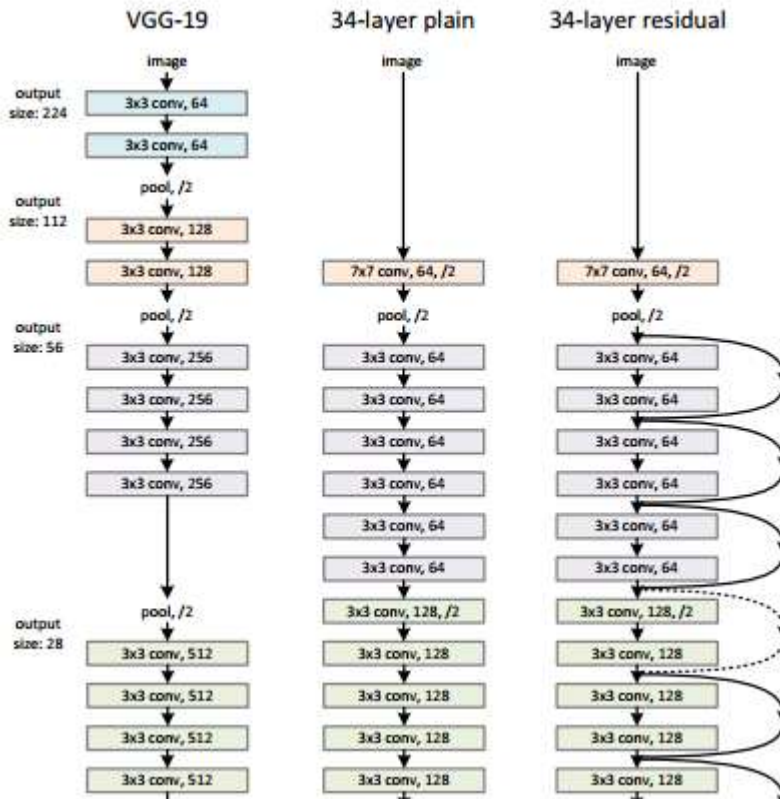


Figure B.2. Plain Neural Network vs Residual Network

APPENDIX C: SNAPSHOTS

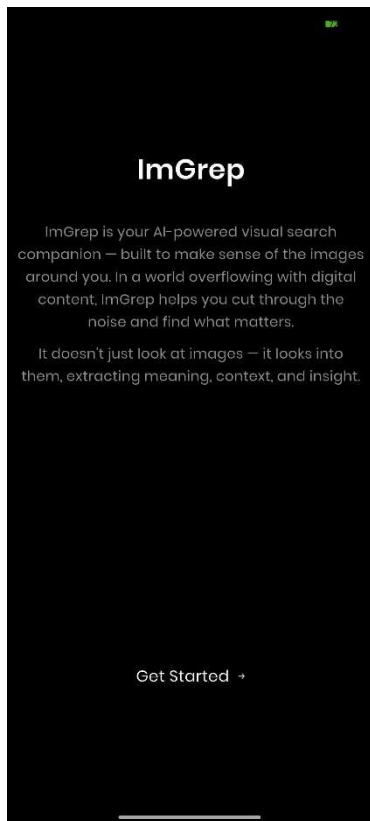


Figure C.1. Get Started Page



Figure C.2. Library Page



Figure C.3. Metadata and Caption View Page



Figure C.4. Image Viewer Page



Figure C.5. Search Page